Rolepilot AI Model Selection Report

Why Rolepilot's AI test-generation and evaluation pipeline is built on the Google Gemma 3 model family, how a single model design scales across four model sizes and six hardware profiles, and the exact sampling/configuration parameters used for every Ollama call in the codebase.

PROJECT

Rolepilot — FastAPI backend, Next.js frontend, PostgreSQL, local AI via Ollama.

MODEL FAMILY

Google Gemma 3 — `gemma3:1b`, `gemma3:4b`, `gemma3:12b`, `gemma3:27b`.

SELECTION MECHANISM

`start.sh` auto-detects GPU VRAM / CPU / RAM and writes `backend/.env.hardware` with the matching profile.

RUNTIME

Ollama `/api/generate`, JSON mode, single persistent HTTP client, retry with exponential backoff.

Gemma 3 was selected because one model family, with one prompt design, covers the entire range from a CPU-only fallback machine to a 16GB+ VRAM GPU server — while reliably returning valid, schema-conformant JSON.

Prepared by: A. M. Rafinul Huq | Date: June 15, 2026

Generated from inspection of the Rolepilot repository: `start.sh`, `backend/app/config.py`, `backend/app/services/ollama_service.py`, `backend/app/services/test_generation_service.py`, `backend/app/services/evaluation_service.py`, `backend/app/services/skill_extraction_service.py`, and `backend/.env.hardware`.

1. Executive Summary

> Rolepilot uses Google's **Gemma 3** model family, served locally through **Ollama**, for three AI-driven jobs: extracting structured skills from a job description, generating three full assessment tests (Technical, Reasoning, Problem Solving) from those skills, and scoring open-ended candidate answers during evaluation. All three jobs depend on the model returning **valid JSON that matches a Pydantic schema** — not free-form prose — so model choice was driven primarily by JSON reliability, not just raw benchmark scores.
>
> Gemma 3 ships in four sizes — 1B, 4B, 12B, and 27B parameters — all sharing the same prompt format, system-prompt conventions, and JSON-mode behavior in Ollama. This lets Rolepilot run the *identical* generation and evaluation code on hardware ranging from a 4GB laptop GPU up to a dedicated 16GB+ VRAM server, simply by changing the `OLLAMA_MODEL` environment variable. `start.sh` automates this choice via a six-tier hardware profile table (Section 3.2).

2. Why Gemma 3

2.1 Reliable structured JSON output at every size

Every Ollama call in Rolepilot sets `"format": "json"`, which constrains the model's token sampling to always emit syntactically valid JSON. Gemma 3 follows this constraint well even at the smallest 1B size, and the resulting JSON consistently matches the expected shape (question lists, option keys, scores) closely enough that `GeneratedTestAI` / `MCQQuestionAI` / `OpenEndedQuestionAI` Pydantic models validate it on the first or second attempt in the large majority of cases. The remaining cases are caught by a bounded "correction prompt" retry loop (`MAX_CORRECTION_ATTEMPTS = 3`, see Section 3.1).

2.2 One family, four sizes, zero pipeline changes

Because `gemma3:1b`, `gemma3:4b`, `gemma3:12b`, and `gemma3:27b` all behave consistently under the same system prompt and JSON-mode settings, Rolepilot does not need per-model prompt variants. The hardware-detection script (`start.sh`) simply substitutes the model name and adjusts concurrency (`OLLAMA_NUM_PARALLEL`), thread count, and per-request timeout. This is what makes the six-profile table in Section 3.2 possible with a single codebase.

2.3 Fully local, fully offline inference via Ollama

Gemma 3 has first-class Ollama packaging (`ollama pull gemma3:<size>`), so the entire AI pipeline runs on the same machine as the backend — no API keys, no per-token cost, and no job-description or candidate-answer text ever leaves the server. `OLLAMA_KEEP_ALIVE=-1` keeps the model resident in VRAM/RAM between requests so there is no reload penalty between the skill-extraction, generation, and evaluation calls that make up a single workflow.

2.4 Strong instruction-following at small parameter counts

The default and most common configuration is `gemma3:4b` (the "medium" profile, matching the 4GB-VRAM development machine this project targets day to day). At 4B parameters, Gemma 3 reliably produces on-topic Technical / Reasoning / Problem-Solving questions, plausible distractor options for MCQs, and usable open-ended grading rationale — output quality that was not realistic with older small models, and which is what makes the "medium" and "low" hardware tiers viable at all.

2.5 Headroom to scale up without re-architecting

On machines with 8GB or more VRAM, Rolepilot switches to `gemma3:12b` or `gemma3:27b` and raises `OLLAMA_NUM_PARALLEL` so the three test types can be generated concurrently instead of sequentially. The same prompts, the same JSON schemas, and the same retry/correction logic apply unchanged — only the model name,

parallelism, and timeout budget change. This means a hardware upgrade alone (no code changes) is enough to move from the "medium" profile to "high" or "ultra".

3. Generation Configuration

3.1 Sampling parameters

These parameters are set on every `OllamaClient.generate()` call (`backend/app/services/ollama_service.py`), with per-call seed values layered on top for reproducibility and controlled variation across retries.

Parameter	Value	Purpose
`format`	`"json"`	Ollama JSON mode — constrains decoding so the response is always syntactically valid JSON, required for Pydantic schema validation.
`temperature`	`0.1`	Low randomness for structured, repeatable output. Used for all generation, extraction, and scoring calls.
`num_predict`	`8192`	Maximum tokens per response — enough headroom for a full test (5 MCQ + 2 open-ended questions) in one call.
`seed` — test generation (initial)	`101` / `202` / `303`	Fixed per test type (Technical / Reasoning / Problem Solving) so each test type is deterministic and distinct from the others.
`seed` — test generation (correction retry)	`seed + attempt × 13`	Each correction-prompt retry (up to `MAX_CORRECTION_ATTEMPTS = 3`) uses a new seed derived from the base seed, so a retry is not just a repeat of the same failed output.
`seed` — single-test regeneration	`random.randint(1000, 9999)`	When an admin regenerates one test, a random seed is used deliberately so the new version differs from the original.
`seed` — skill extraction retry	`attempt × 7 + 42`	Varies the seed on each retry of JD skill extraction (`skill_extraction_service.py`).
`seed` — open-ended answer scoring	`attempt × 23 + 9` and `attempt × 17 + 5`	Two independent seed sequences used during open-ended evaluation retries (`evaluation_service.py`).
`seed` — aggregate AI recommendation	`99` (fixed)	Fixed seed for the final per-attempt hiring recommendation summary, so the same attempt always produces the same overall verdict text.
`seed` — evaluation correction retry	`attempt × 31 + 7`	Additional retry seed sequence used elsewhere in `evaluation_service.py`.

3.2 Per-hardware-profile configuration

`start.sh` detects GPU VRAM (via `nvidia-smi`), CPU core count, and total system RAM, then selects exactly one of six profiles below and writes the result to `backend/.env.hardware`. "Threads" is the value assigned to `OLLAMA_NUM_THREADS`; "Half" means `CPU cores ÷ 2`, "All" means every detected CPU core. "Sequential" reflects `TEST_SEQUENTIAL_GENERATION` — when `true`, the three tests are generated one at a time instead of concurrently.

Profile	Hardware trigger	Model	MCQ	OE	Parallel	Threads	Timeout	Sequential
`ultra`	GPU, VRAM ≥ 16 GB	`gemma3:27b`	5	2	3	Half	900s	No
`high`	GPU, VRAM ≥ 8 GB	`gemma3:12b`	5	2	2	Half	600s	No
`medium`	GPU, VRAM ≥ 4 GB	`gemma3:4b`	5	2	1	Half	600s	Yes
`low`	GPU, VRAM ≥ 2 GB	`gemma3:1b`	3	1	1	Half	300s	Yes
`cpu_fast`	No usable GPU, RAM ≥ 16 GB	`gemma3:4b`	3	1	1	All	900s	Yes
`cpu_slow`	No usable GPU, RAM < 16 GB	`gemma3:1b`	2	1	1	All	1200s	Yes

Note: a GPU with less than 2GB VRAM is treated as "no usable GPU" and falls through to the `cpu_fast` / `cpu_slow` rows based on total system RAM.

3.3 Additional global Ollama settings

These are set once when `start.sh` launches the native Ollama server (not inside Docker, so it has direct GPU access), or come from `backend/app/config.py` defaults.

Setting	Value	Purpose
`OLLAMA_HOST`	`0.0.0.0:11434`	Binds Ollama to all interfaces so the Dockerized backend can reach the host-native Ollama process.
`OLLAMA_KEEP_ALIVE`	`-1`	Keeps the loaded model resident indefinitely — avoids reload latency between the skill-extraction, generation, and evaluation calls of one workflow.
`OLLAMA_FLASH_ATTENTION`	`1`	Enables flash attention for faster inference and lower memory use.
`OLLAMA_MAX_LOADED_MODELS`	`1`	Only one model resident at a time — avoids VRAM contention on small GPUs.
`OLLAMA_NUM_PARALLEL`	1–3 (by profile)	Number of requests Ollama will batch/serve concurrently; see Section 3.2.
`ollama_timeout_seconds`	300–1200s (by profile)	Per-request HTTP read timeout for `/api/generate`; default `600` in `config.py`, overridden per profile.
`ollama_max_retries`	`3`	HTTP-level retries with exponential backoff (2s → 4s → 8s) on transient connection/timeout failures. Does not retry on 4xx errors.
`MAX_CORRECTION_ATTEMPTS`	`3`	Application-level retries that resend the prompt with a "correction" instruction when the JSON response fails Pydantic validation.

4. Sources

- Rolepilot repository: `start.sh` (hardware detection and profile table).
- Rolepilot repository: `backend/app/config.py` (Ollama and test-generation defaults).
- Rolepilot repository: `backend/app/services/ollama_service.py` (Ollama HTTP client, JSON-mode payload, retry logic).
- Rolepilot repository: `backend/app/services/test_generation_service.py` (test generation pipeline, seeds, correction retries).
- Rolepilot repository: `backend/app/services/evaluation_service.py` (open-ended scoring and aggregate recommendation seeds).
- Rolepilot repository: `backend/app/services/skill_extraction_service.py` (skill extraction retry seeds).
- Rolepilot repository: `backend/.env.hardware` (example generated profile on the current development machine).